



University of Central Punjab

Compiler Construction

Project Phase 2

VigoLang

Syntax Analyzer

Student Name: Muhammad Ahmad

Registration: L1F22BSCS0634

Section: G1

Teacher: Sir Asif Farooq

Contents

1	Language Overview	2
1.1	Introduction	2
1.2	Language Features	2
1.3	Key Constructs	2
2	Context-Free Grammar (CFG)	3
2.1	Program Structure	3
2.2	Variable Declarations	3
2.3	Function and Class Declarations	3
2.4	Statements	3
2.5	Control Flow	3
2.6	I/O and Control Statements	4
2.7	Expressions	4
3	FIRST and FOLLOW Sets	5
3.1	FIRST Sets	5
3.2	FOLLOW Sets	5
4	Parse Tree	6
4.1	Sample Program	6
4.2	Parse Tree Diagram	6
5	Phase 1 Token Integration	7
5.1	How Tokens are Connected	7
5.2	Keywords from Phase 1	7
5.3	Operators and Punctuation from Phase 1	7
5.4	Compilation Flow	8
6	Conclusion	9

1 Language Overview

1.1 Introduction

VigoLang is a satirical, Urdu-inspired programming language that uses Pakistani political terminology as programming constructs. This is purely an academic exercise for learning compiler construction.

1.2 Language Features

- **20 Keywords:** Political terminology mapped to programming constructs
- **9 Operators:** Custom operators like `+:, -:,*:,:=,=?,>?,<?`
- **6 Punctuation:** `@` (terminator), `{ { }` (blocks), `[ [ ] ]` (expressions)
- **Identifier Prefix:** All variables must start with caret (^)

1.3 Key Constructs

Construct	VigoLang Keyword	Equivalent
Main function	ghq	main
If condition	order_hai	if
Else if	doosra_order	else if
Else	warna_vigo	else
For loop	long_march	for
While loop	jab_tak_missing	while
Output	farmaan	print
Input	taftish	input
Return	nro	return
Break	deal_ho_gai	break
Continue	chaltay_raho	continue

2 Context-Free Grammar (CFG)

2.1 Program Structure

$$\begin{aligned}
 \langle \text{Program} \rangle &\rightarrow \langle \text{DeclList} \rangle \langle \text{MainFunc} \rangle \\
 \langle \text{MainFunc} \rangle &\rightarrow \text{ghq} \text{ } [[\text{ }]] \{ \{ \langle \text{StmtList} \rangle \} \} \\
 \langle \text{DeclList} \rangle &\rightarrow \langle \text{Declaration} \rangle \langle \text{DeclList} \rangle \mid \varepsilon \\
 \langle \text{Declaration} \rangle &\rightarrow \langle \text{VarDecl} \rangle \mid \langle \text{FuncDecl} \rangle \mid \langle \text{ClassDecl} \rangle
 \end{aligned}$$

2.2 Variable Declarations

$$\begin{aligned}
 \langle \text{VarDecl} \rangle &\rightarrow \langle \text{Type} \rangle \langle \text{ID} \rangle @ \\
 &\mid \langle \text{Type} \rangle \langle \text{ID} \rangle := \langle \text{Expr} \rangle @ \\
 &\mid \text{ain} \langle \text{Type} \rangle \langle \text{ID} \rangle := \langle \text{Expr} \rangle @ \\
 \langle \text{Type} \rangle &\rightarrow \text{qaidi_no} \mid \text{float_sarkar} \mid \text{bayania} \mid \text{ishara} \mid \text{namaloom}
 \end{aligned}$$

2.3 Function and Class Declarations

$$\begin{aligned}
 \langle \text{FuncDecl} \rangle &\rightarrow \langle \text{Type} \rangle \langle \text{ID} \rangle [[\langle \text{ParamList} \rangle]] \{ \{ \langle \text{StmtList} \rangle \} \} \\
 \langle \text{ParamList} \rangle &\rightarrow \langle \text{Param} \rangle \langle \text{ParamList}' \rangle \mid \varepsilon \\
 \langle \text{Param} \rangle &\rightarrow \langle \text{Type} \rangle \langle \text{ID} \rangle \\
 \langle \text{ClassDecl} \rangle &\rightarrow \text{safehouse} \langle \text{ID} \rangle \{ \{ \langle \text{MemberList} \rangle \} \} @ \\
 \langle \text{MemberList} \rangle &\rightarrow \langle \text{VarDecl} \rangle \langle \text{MemberList} \rangle \mid \varepsilon
 \end{aligned}$$

2.4 Statements

$$\begin{aligned}
 \langle \text{StmtList} \rangle &\rightarrow \langle \text{Statement} \rangle \langle \text{StmtList} \rangle \mid \varepsilon \\
 \langle \text{Statement} \rangle &\rightarrow \langle \text{AssignStmt} \rangle \mid \langle \text{IfStmt} \rangle \mid \langle \text{LoopStmt} \rangle \\
 &\mid \langle \text{OutputStmt} \rangle \mid \langle \text{InputStmt} \rangle \mid \langle \text{ReturnStmt} \rangle \\
 &\mid \langle \text{BreakStmt} \rangle \mid \langle \text{ContinueStmt} \rangle \mid \langle \text{FuncCall} \rangle \\
 \langle \text{AssignStmt} \rangle &\rightarrow \langle \text{ID} \rangle := \langle \text{Expr} \rangle @
 \end{aligned}$$

2.5 Control Flow

$$\begin{aligned}
 \langle \text{IfStmt} \rangle &\rightarrow \text{order_hai} [[\langle \text{Condition} \rangle]] \{ \{ \langle \text{StmtList} \rangle \} \} \langle \text{ElseIfPart} \rangle \langle \text{ElsePart} \rangle \\
 \langle \text{ElseIfPart} \rangle &\rightarrow \text{doosra_order} [[\langle \text{Condition} \rangle]] \{ \{ \langle \text{StmtList} \rangle \} \} \langle \text{ElseIfPart} \rangle \mid \varepsilon \\
 \langle \text{ElsePart} \rangle &\rightarrow \text{warna_vigo} \{ \{ \langle \text{StmtList} \rangle \} \} \mid \varepsilon \\
 \langle \text{LoopStmt} \rangle &\rightarrow \langle \text{ForLoop} \rangle \mid \langle \text{WhileLoop} \rangle \\
 \langle \text{ForLoop} \rangle &\rightarrow \text{long_march} [[\langle \text{AssignStmt} \rangle \langle \text{Condition} \rangle @ \langle \text{ID} \rangle := \langle \text{Expr} \rangle]] \\
 &\quad \{ \{ \langle \text{StmtList} \rangle \} \} \\
 \langle \text{WhileLoop} \rangle &\rightarrow \text{jab_tak_missing} [[\langle \text{Condition} \rangle]] \{ \{ \langle \text{StmtList} \rangle \} \}
 \end{aligned}$$

2.6 I/O and Control Statements

$$\begin{aligned}
\langle \text{OutputStmt} \rangle &\rightarrow \text{farmaan} << \langle \text{Expr} \rangle @ \\
\langle \text{InputStmt} \rangle &\rightarrow \text{taftish} >> \langle \text{ID} \rangle @ \\
\langle \text{ReturnStmt} \rangle &\rightarrow \text{nro} \langle \text{Expr} \rangle @ \\
\langle \text{BreakStmt} \rangle &\rightarrow \text{deal_ho_gai} @ \\
\langle \text{ContinueStmt} \rangle &\rightarrow \text{chaltay_raho} @ \\
\langle \text{FuncCall} \rangle &\rightarrow \langle \text{ID} \rangle [[\langle \text{ArgList} \rangle]] @ \\
\langle \text{ArgList} \rangle &\rightarrow \langle \text{Expr} \rangle \langle \text{ArgList}' \rangle \mid \varepsilon
\end{aligned}$$

2.7 Expressions

$$\begin{aligned}
\langle \text{Condition} \rangle &\rightarrow \langle \text{Expr} \rangle \langle \text{RelOp} \rangle \langle \text{Expr} \rangle \\
\langle \text{RelOp} \rangle &\rightarrow =? \mid >? \mid <? \\
\langle \text{Expr} \rangle &\rightarrow \langle \text{Term} \rangle \langle \text{Expr}' \rangle \\
\langle \text{Expr}' \rangle &\rightarrow +: \langle \text{Term} \rangle \langle \text{Expr}' \rangle \mid -: \langle \text{Term} \rangle \langle \text{Expr}' \rangle \mid \varepsilon \\
\langle \text{Term} \rangle &\rightarrow \langle \text{Factor} \rangle \langle \text{Term}' \rangle \\
\langle \text{Term}' \rangle &\rightarrow *: \langle \text{Factor} \rangle \langle \text{Term}' \rangle \mid \varepsilon \\
\langle \text{Factor} \rangle &\rightarrow \langle \text{ID} \rangle \mid \langle \text{Number} \rangle \mid \langle \text{String} \rangle \mid \text{neutral} \mid \text{janwar}
\end{aligned}$$

3 FIRST and FOLLOW Sets

3.1 FIRST Sets

Non-Terminal	FIRST Set
Statement	{ [^] , order_hai, long_march, jab_tak_missing, farmaan, taftish, nro, deal_ho_gai, chaltay_rahoo}
Expr	{ [^] , NUMBER, STRING, neutral, janwar}
Type	{qaidi_no, float_sarkar, bayania, ishara, namaloom}
IfStmt	{order_hai}
LoopStmt	{long_march, jab_tak_missing}
OutputStmt	{farmaan}
InputStmt	{taftish}

Table 1: FIRST Sets for VigoLang Non-Terminals

3.2 FOLLOW Sets

Non-Terminal	FOLLOW Set
Statement	{FIRST(Statement), }}}
Expr	{@, =?, ¿?, ¡?,], , }
StmtList	{}}
Type	{ [^] }
Condition	{}}
ElsePart	{FIRST(Statement), }}}

Table 2: FOLLOW Sets for VigoLang Non-Terminals

4 Parse Tree

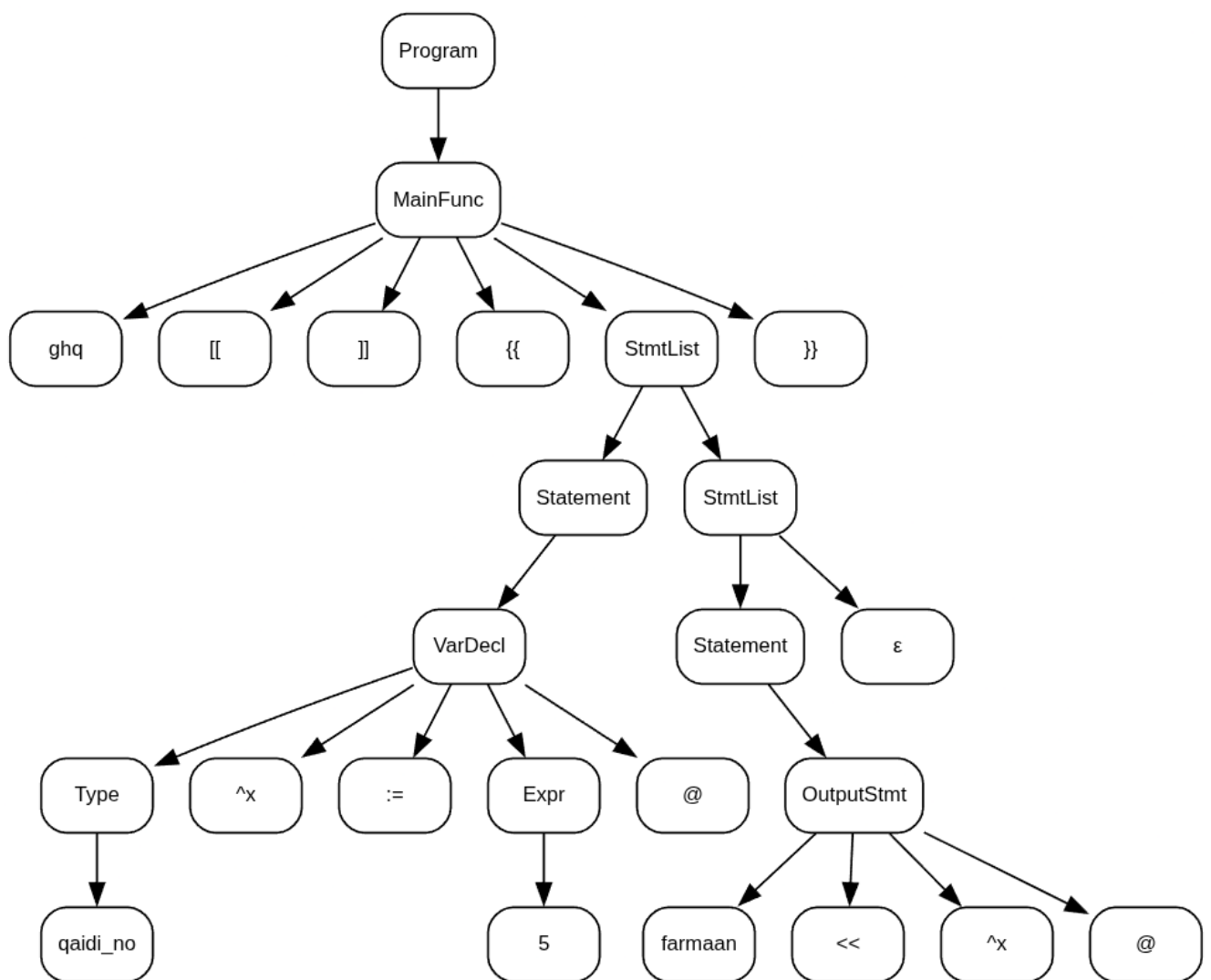
4.1 Sample Program

```

1 ghq [[ ]] {{
2   qaidi_no ^x := 5@
3   farmaan << ^x@
4 }}

```

4.2 Parse Tree Diagram



5 Phase 1 Token Integration

5.1 How Tokens are Connected

The parser reuses all tokens from Phase 1 lexical analyzer:

1. **Token Declaration:** All tokens from scanner.l are declared in parser.y using `%token`
2. **Header Sharing:** scanner.l includes `y.tab.h` generated by yacc
3. **Token Return:** Scanner returns token values to parser instead of printing to file

5.2 Keywords from Phase 1

Keyword	Purpose	Token Name
ghq	Main function	KEYWORD_GHQ
safehouse	Class	KEYWORD_SAFEHOUSE
order_hai	If	KEYWORD_ORDER_HAI
doosra_order	Else if	KEYWORD_DOOSRA_ORDER
warna_vigo	Else	KEYWORD_WARNA_VIGO
long_march	For loop	KEYWORD_LONG_MARCH
jab_tak_missing	While loop	KEYWORD_JAB_TAK_MISSING
farmaan	Output	KEYWORD_FARMAAN
taftish	Input	KEYWORD_TAFTISH
nro	Return	KEYWORD_NRO
deal_ho_gai	Break	KEYWORD_DEAL_HO_GAI
chaltay_raho	Continue	KEYWORD_CHALTAY_RAHQ
qaidi_no	Integer	KEYWORD_QAIDI_NO
ain	Constant	KEYWORD_AIN
neutral	True	KEYWORD_NEUTRAL
janwar	False	KEYWORD_JANWAR

5.3 Operators and Punctuation from Phase 1

Symbol	Purpose	Token Name
<code>:=</code>	Assignment	OP_ASSIGN
<code>+: -: *:</code>	Arithmetic	OP_ADD, OP_SUB, OP_MUL
<code>=? <? >?</code>	Comparison	OP_EQUAL, OP_GREATER, OP_LESS
<code>ii ii</code>	I/O Stream	OP_OUTPUT, OP_INPUT
@	Terminator	PUNCT_TERMINATOR
{ { } }	Block	PUNCT_BLOCK_START/END
[[]]	Expression	PUNCT_EXPR_START/END
,	Separator	PUNCT_COMMA

5.4 Compilation Flow

```
1 flex scanner.l          # Creates lex.yy.c
2 yacc -d parser.y        # Creates y.tab.c and y.tab.h
3 gcc lex.yy.c y.tab.c -o vigolang_parser -lfl
4 ./vigolang_parser test_program.vigo
```

6 Conclusion

The Phase 2 syntax analyzer successfully validates VigoLang programs:

- Uses all 20 keywords from Phase 1
- Uses all 9 operators from Phase 1
- Uses all 6 punctuation symbols from Phase 1
- Complete CFG covering all mandatory constructs
- Error handling with line numbers and token information

Mandatory Constructs Covered:

1. Program start/end (`ghq`)
2. Variable declaration (`qaidi_no`, `bayania`, etc.)
3. Assignment statement (`:=`)
4. Conditional statement (`order_hai`, `doosra_order`, `warna_vigo`)
5. Loop statement (`long_march`, `jab_tak_missing`)
6. Input/Output statement (`farmaan`, `taftish`)